

Scala :

Scala (Scalable Language) is a programming language that runs on the **JVM (Java Virtual Machine)** and combines:

Variables in scala:

There are two types of variables :

- **Val (Immutable)**
- **Var (Mutable)**

It automatically detects the type of variable (Type Inference).

Val (Immutable):

```
scala> val name="jeeva"
val name: String = jeeva

scala> name="ram"
      ^
error: reassignment to val
```

Var (Mutable):

```
scala> var name="java"
var name: String = java

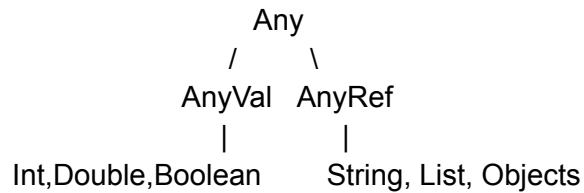
scala> name="ram"
// mutated name

scala> name
val res0: String = ram

scala> name=3
      ^
error: type mismatch;
 found   : Int(3)
 required: String
```

Not able to change the data type of the variable, once assigned.

Scala type hierarchy:



Assigning Block Expression :

In Java or C++ code block is a list of statements in a curly braces {}

In Scala, a {} block is a list of expression, and result is also an expression

The Value of a block is the value of the last expression of it

```
scala> val x={val a=10;val b=100;b-a}
val x: Int = 90
```

If-else:

Semicolons are optional in scala

```
scala> var x=5
var x: Int = 5
```

```
scala> val s=if(x>0 && x<6) 1 else 0
val s: Int = 1
```

```
scala> val s=if(x>0 && x<6) "positive" else 0
val s: Any = positive
```

Every expression in a scala has an type

First If statement has type int

Second statement has type Any.

Type of a mixed expression(String,Int) is supertype(Any) of both branches.

While loop:

```
scala> var a="5000"  
var a: String = 5000  
  
scala> var i=0  
var i: Int = 0  
  
scala> while(i<a.length){  
  | print(a(i))  
  | i+=1  
  | }  
5000
```

++i or i++,--i or i-- operators not works on scala

Foreach Loop:

```
scala> a.foreach(x=>println(x))  
5  
0  
0  
0  
  
scala> a.foreach(println)  
5  
0  
0  
0
```

For loop:

```
scala> for(i<- 1 to 5) // 1 to 5(included)  
  | print(i)  
12345  
  
scala> for (i<- 0 until in.length) // 0 until 11(excluded)  
  | sum+=i
```

```
scala> print(sum)
55
```

to - included
until - excluded

Nested for :

```
scala> for(i<- 1 to 3)
  | for(j<- 1 to 3)println(i*10+j)
11
12
13
21
22
23
31
32
33
```

Multi generators:

```
scala> for(i<- 1 to 3;j<- 1 to 3)
  | println(i*10+j)
11
12
13
21
22
23
31
32
33
```

if inside a loop:

```
scala> for(i<-1 to 3;j<-1 to 3;if i==j)
  | println(10*i+j)
11
22
33
```

variables inside a loop:

```
scala> for(i<-1 to 3;j<-1 to 3;if i==j)
  | println(10*i+j)
11
22
33
```

if the for loop start with yield, it returns collection of values:

```
scala> val x=for(i<- 1 to 5)yield i*i
val x: IndexedSeq[Int] = Vector(1, 4, 9, 16, 25)

scala> for(i<-x)println(i)
1
4
9
16
25
```

Functions:

```
scala> def area(radius:Int)={3.14*radius*radius}
def area(radius: Int): Double

scala> area(50)
val res22: Double = 7850.0
```

There is no need of a Return statement in Scala function

We need to specify the datatype for recursive function

```
scala> def fact(n:Int):Int=if (n==0) 1 else n*fact(n-1)
def fact(n: Int): Int

scala> fact(5)
val res21: Int = 120
```

Area of Rectangle:

```
scala> def area(l:Int,b:Int)={l*b}
def area(l: Int, b: Int): Int
```

```
scala> area(20,30)
val res23: Int = 600
```

Collections:

Scala has a rich library of Collections,

- Array
- ArrayBuffers
- Maps
- Tuples
- Lists

Array:

```
scala> val n=new Array[Int](10)
val n: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
```

```
scala> val n=Array("Hello","World")
val n: Array[String] = Array(Hello, World)
```

```
scala> val n=new Array[String](10)
val n: Array[String] = Array(null, null, null, null, null, null, null, null, null, null)
```

ArrayBuffer:

```
scala> import scala.collection.mutable.ArrayBuffer
import scala.collection.mutable.ArrayBuffer
```

```
scala> val a=ArrayBuffer[Int]()
val a: scala.collection.mutable.ArrayBuffer[Int] = ArrayBuffer()
```

```
scala> a+=Array(2,3,4)
val res10: a.type = ArrayBuffer(2, 3, 4)
```

```
scala> a+=Array(2,3,4)
val res11: a.type = ArrayBuffer(2, 3, 4, 2, 3, 4)

scala> a+=Array(2,3,5)
val res12: a.type = ArrayBuffer(2, 3, 4, 2, 3, 4, 2, 3, 5)

scala> Array(1,2,3,4).sum
val res1: Int = 10

scala> Array(1,5,6,9).max
val res2: Int = 9

scala> val a=Array(1,7,2,9)
val a: Array[Int] = Array(1, 7, 2, 9)

scala> scala.util.Sorting.quickSort(a)

scala> a.mkString("***")
val res4: String = 1**2**7**9
```

Map:

```
scala> val mapping=Map("Vishal"->"Kumar","Vijay"->"Verma")
val mapping: scala.collection.immutable.Map[String,String] = Map(Vishal -> Kumar, Vijay -> Verma)

scala> val mapping=scala.collection.mutable.Map("Vishal"->"K","Vijay"->"V")
val mapping: scala.collection.mutable.Map[String,String] = HashMap(Vijay -> V, Vishal -> K)

scala> val x=mapping("Vishal")
val x: String = K

scala> val x=mapping.getOrElse("Vish",0)
val x: Any = 0

scala> mapping-="Vishal"
val res13: mapping.type = HashMap(Vijay -> V)

scala> mapping+=("Ajay"->"Sharma")
val res14: mapping.type = HashMap(Vijay -> V, Ajay -> Sharma)
```

Tuple:

```
scala> val a=(1,4,"Bob","Jack")  
val a: (Int, Int, String, String) = (1,4,Bob,Jack)
```

Offset(index) starts with 1

```
scala> a._2  
val res15: Int = 4
```

```
scala> "New Delhi India".partition(_._isUpper)  
val res18: (String, String) = (NDI,ew elhi ndia)
```

List:

```
scala> val lst=List(1,3, 4)  
val lst: List[Int] = List(1, 3, 4)
```

```
scala> lst.head  
val res22: Int = 1
```

```
scala> lst.tail  
val res23: List[Int] = List(3, 4)
```

```
scala> 2::List(4,5)  
val res24: List[Int] = List(2, 4, 5)
```

```
scala> def sum(l:List[Int]):Int={if (l==Nil)0 else l.head+sum(l.tail)}  
def sum(l: List[Int]): Int
```

```
scala> var l=List(1,2,3,4)  
var l: List[Int] = List(1, 2, 3, 4)
```

```
scala> sum(l)  
val res28: Int = 10
```

Classes in Scala:

```
scala> class Ctr{  
  | private var value=0
```



```
| def incr()={value+=1}  
| def curr()=value  
| }  
class Ctr
```

In scala class is not defined as a public

A source file can contain multiple classes

All of the classes could be public

```
scala> val ctr1=new Ctr  
val ctr1: Ctr = Ctr@248ad8a1
```

```
scala> ctr1.curr()  
val res11: Int = 0
```

```
scala> ctr1.incr()
```

```
scala> ctr1.curr()  
val res13: Int = 1
```

public variable:

```
scala> class Student{  
| var age=25}  
class Student
```

```
scala> val s1=new Student  
val s1: Student = Student@295d8852
```

```
scala> s1.age  
val res14: Int = 25
```

```
scala> s1.age=30  
// mutated s1.age
```

```
scala> s1.age  
val res15: Int = 30
```

Setters and Getters:

```
scala> class Student{  
| private var privateAge=0
```

```

    | def age=privateAge
    | def age_=(newAge:Int)={if(newAge>privateAge)privateAge=newAge}
    | }
class Student

scala> var s1=new Student
var s1: Student = Student@64d05e02

scala> s1.age
val res18: Int = 0

scala> s1.age=(1)
// mutated s1.age

scala> s1.age
val res19: Int = 1

```

Constructors:

There are two types of constructors in Scala:

- Auxiliary constructor
- Primary constructor

The auxiliary constructors in scala called this.

Each auxiliary constructor must start with a call to either a previously defined auxiliary constructors or the primary constructor

Auxiliary constructor:

```

scala> class Student{
    | private var weight=0
    | private var age=0
    | def this(w:Int)={
    | this();
    | this.weight=w}
    | def this(w:Int,a:Int)={
    | this(w)
    | this.age=a}
    | def publicAge=age
    | def publicWeight=weight

```

```

    | }
class Student

scala> var s1=new Student()
var s1: Student = Student@7ca29d15

scala> var s2=new Student(50)
var s2: Student = Student@7fb44edb

scala> var s3=new Student(50,25)
var s3: Student = Student@372036d9

scala> println(s1.publicAge,s1.publicWeight)
(0,0)

scala> println(s2.publicAge,s2.publicWeight)
(0,50)

scala> println(s3.publicAge,s3.publicWeight)
(25,50)

```

Primary constructor:

Primary constructor are not defined by method

The parameters for primary constructor are placed immediately after the class name

```

scala> class Student(w:Int,a:Int){
    | private var weight=w
    | private var age=a
    | def publicAge=age
    | def publicWeight=weight
    | }
class Student

scala> var s1=new Student(60,27)
var s1: Student = Student@7c693055

scala> print(s1.publicWeight,s1.publicAge)
(60,27)

```

Nested Class:

```
scala> import scala.collection.mutable.ArrayBuffer;class Class{
  | class Student(id:Int){
  |   val contacts=new ArrayBuffer[Student]
  | }
  | private val students=new ArrayBuffer[Student]
  | def join(id:Int)={
  |   students+=new Student(id)
  | }
import scala.collection.mutable.ArrayBuffer
class Class

scala> val c1=new Class
val c1: Class = Class@22d23c27

scala> var a=c1.join(23)
var a: scala.collection.mutable.ArrayBuffer[c1.Student] =
ArrayBuffer(Class$Student@13157620)

scala> var b=c1.join(24)
var b: scala.collection.mutable.ArrayBuffer[c1.Student] =
ArrayBuffer(Class$Student@13157620, Class$Student@5e8633da)
```

Sigletons:

Not allowed to create a object

```
scala> object Reservation{
  | private var lastNum=0
  | def newReservationNum()={lastNum+=1;lastNum}
  | }
object Reservation

scala> Reservation.newReservationNum()
val res9: Int = 1

scala> Reservation.newReservationNum()
val res10: Int = 2

scala> var obj=new Reservation
      ^
```

error: not found: type Reservation

Packages:

To add the items to a package, they can be included in package statement

```
package com{  
  
    package spark{  
  
        package mytech{  
  
            class Emp  
  
            ...  
        }  
    }  
}
```

The class Emp can be accessed from anywhere as com.spark.mytech.Emp

A single file can have more than one package

Scope Rules:

Member names could ne accessed from enclosing scope

```
package com{  
  
    package test1{  
  
        object Utils{  
  
            def sayHello="Hello"  
  
        }  
    }  
  
    package test2{  
  
        class Test{
```

```

        def sayHi()={
            Utils.sayHello
        }
    }
}

```

Package visibility:

In Scala it is achieved through qualifiers

```

package com.spark.mytech

class Employee{

    private[mytech] def sayHi="Hi"
}

```

Imports and Implicit Imports

Packages/classes can be imported in Scala

To use short names instead of long ones

Extending a class:

classes can be extended using **extends** keyword.

```

scala> class Animal{
    | print("Animal")
    | }
class Animal

scala> class Dog extends Animal{
    | print("Dog")
    | }
class Dog

```

Cannot inherit from final class.

```

scala> final class Animal{

```

```

    | print("Animal")
    |}
class Animal

```

```

scala> class Dog extends Animal{
    | print("Dog")
    |}

```

^

error: illegal inheritance from final class Animal

Superclass methods or fields could be overridden in subclasses

Overriding Methods:

Overriding modifier must be used to override an abstract method:

```

public class Dog{
    ...
    override def bark={"woof"}
    ...
}

```

Superclass Constructor:

```

class Emp(name:String,dept,salary:Double) extends person(name,dept){
    ...
}

```

Abstract Classes:

```

abstract class Emp(name:String){
    def id:Int
}

```

In subclass, we don't need to specify override while defining an abstract superclass method

```

class MyTechEmp(name:String) extends Emp(name){
    def id=name.hashCode // no override required
}

```

